API and NGINX

Tags	em7ngx.access.log, em7ngx_web.ui.conf, ftr_interface.inc, em7ngx.access.log, REQUEST_URI, segfault, ftr_data_performance.inc
Team Owner	TBD
Status	ACTIVE
Last Modified	Mar 12, 2021

- Introduction
- Diagnosis and Mitigation
  - segfaults in php-fpm
  - long running queries involving master._tmp_pres_to_labelob
  - Viewing a log of API hits
    - pre 11.2, access logs
    - post 11.2, with associated queries, EM-43012
  - Badly Performing API queries
- Theory
  - php code: G3code/gui/is/classes/features/ftr_interface.inc
- Post Mortems
  - 2024-11-04 EM-71013: SL Data Performance API issue
  - 2023-02-21 EM-56998 segfaults in php-fpm caused by out of memory
  - 2023-02-13 EM-56909 API Long Running Query /api/data_performance_raw/device/dynamic_app
  - 2022-12-06 EM-54861 rest api, nginx, ssl, http vs https
  - 2019-07-08 SOL_16338 rest api call generated long running query

Introduction

Diagnosis and Mitigation

segfaults in php-fpm

See: API and NGINX | 2023 02 21 EM 56998 segfaults in php fpm caused by out of memory

long running queries involving master._tmp_pres_to_labelob

These are queries generated by API code.

Here is an example:

```
SELECT ld.id, ld.id 'device',daim.ind 'idx',label.data 'idx_label',labelob.label_obj_id 'label_obj_id',labelob.coll_obj_id 'coll_obj_id',daim.ind 'ind'

FROM master_dev.legend_device ld

LEFT JOIN master.dynamic_app_index_map daim ON (ld.id = daim.did and daim.app_id=1843)

LEFT JOIN dynamic_app_data_1843.app_label label ON (label.did = ld.id AND label.ind=daim.ind)

LEFT JOIN master._tmp_pres_to_labelob labelob ON (label.object=labelob.label_obj_id)

WHERE (ld.id IN ('68041','68043','83333','22663','22664','22665','22666','22667','22668','22669','22670','22671','22672','22673','22674','22675','22676','22677','22678','22679','22680','22681','24962','24963','24965','24966','25676','25677','25680','25682','25688','25690','25692','25694','25784','25786','25787','25788','25789','44109','44111','44112','44114','44119','44120','44123','44126','44127','44129','44131','44132','45008','45009','54054','54055','54056','54065','54066','54067','54524','54525','54526','54527','54528','54529','54530','54905','54906','54907','54908','54909','54910','54911','54912','54913','55303','55304','55305','55306','55307','55308','55309','55310','55311','55312','55313','56915','56916','56917','56918','56919','56920','56921','56922','56923','56924','56925','56926','56927','56928','56929','56930','56931','56932','57009','57010','57011','57012','57013','57014','57015','57016','57017','57018','57019','57020','57021','57022','57023','57024','57025','57026','57027','57028','57252','57253','57254','57262','57263','57264','57290','57291','57292','57293','57294','57295','57296','57297','57298','57299','57300','57301','57327','57328','58898','58899','58900','58901','58902','58903','58904','58905','58906','58907','58908','58909','58910','58911','58912','58913','58914','58915','58916','58917','58918','58919','58920','58921'))

GROUP BY device,coll_obj_id,idx

ORDER BY ld.id asc;
```

They are assembled in the following API functions:

```
./gui/is/classes/features/base_ftr_performance.inc:6:// FIXME_INFO_CLASS: base_ftr_performance  extends base_ftr {
./gui/is/classes/features/base_ftr_performance.inc:371:// FIXME_INFO_FN: _dynapp_init_labeltmp() {
./gui/is/classes/features/base_ftr_performance.inc:386:// FIXME_INFO_FN: } _dynapp_init_labeltmp()
./gui/is/classes/features/base_ftr_performance.inc:635:// FIXME_INFO_CLASS: } base_ftr_performance  extends base_ftr
./gui/is/classes/features/ftr_data_performance.inc:3:// FIXME_INFO_CLASS: ftr_data_performance extends base_ftr_performance {
./gui/is/classes/features/ftr_data_performance.inc:580:// FIXME_INFO_FN: _get_idx_labelspec() {
./gui/is/classes/features/ftr_data_performance.inc:623:// FIXME_INFO_FN: } _get_idx_labelspec()
./gui/is/classes/features/ftr_data_performance.inc:647:// FIXME_INFO_CLASS: } ftr_data_performance extends base_ftr_performance
```

```
./gui/is/classes/features/ftr_data_performance_raw.inc:3:// FIXME_INFO_CLASS: ftr_data_performance_raw extends base_ftr_performance {
./gui/is/classes/features/ftr_data_performance_raw.inc:1159:// FIXME_INFO_FN: _get_idx_labelspec() {
./gui/is/classes/features/ftr_data_performance_raw.inc:1208:// FIXME_INFO_FN: } _get_idx_labelspec()
./gui/is/classes/features/ftr_data_performance_raw.inc:1400:// FIXME_INFO_CLASS: } ftr_data_performance_raw extends base_ftr_performance
```

In this case, they are very long running: API and NGINX | 2023 02 13 EM 56909 API Long Running Query /api/data_performance_raw/device/dyn...

Viewing a log of API hits

pre 11.2, access logs

This file

```
/var/log/em7/em7ngx.access.log
```

can be grepped to show just the API hits like this

```
grep "GET" /var/log/em7/em7ngz.access.log | grep "\/api\/

127.0.0.1 - - [29/Nov/2022:16:55:34 +0000] "GET /api/account/_self/access_hooks HTTP/1.1" 200 11255 "-" "-" "-"
127.0.0.1 - - [29/Nov/2022:16:55:34 +0000] "GET /api/account/_self HTTP/1.1" 200 2020 "-" "-" "-"
127.0.0.1 - - [29/Nov/2022:16:55:34 +0000] "GET /api/organization?hide_filterinfo=1&limit=100000&order._id=asc HTTP/1.1" 200 179 "-" "-" "-"
127.0.0.1 - - [29/Nov/2022:16:55:35 +0000] "GET /api/event?extended_fetch=1&limit=10&offset=0&filter.0.aligned_resource%2Fdevice.in=2%2C1&filter.0.event_policy.in=2532%2C2497%2C2516%2C2569%2C2496%2C2524%2C2528%2C2499%2C2487%2C2519%2C2503%2C2577%2C2624%2C2492%2C2526%2C2554%2C2580%2C2514%2C2510%2C2490%2C2565%2C2483%2C2501%2C2563%2C2567%2C2512%2C2571%2C2559%2C2522&order.date_active=desc&order._id=asc HTTP/1.1" 200 2951 "-" "-" "-"
127.0.0.1 - 1,1,146d55682bc6b69ac082a97d688eb4ca59c14ec084308123e131c2325a89dc59 [29/Nov/2022:17:00:11 +0000] "GET /api/account/_self/access_hooks HTTP/1.1" 200 11255 "-" "-" "-"
127.0.0.1 - 1,1,146d55682bc6b69ac082a97d688eb4ca59c14ec084308123e131c2325a89dc59 [29/Nov/2022:17:00:11 +0000] "GET /api/account/_self HTTP/1.1" 200 2020 "-" "-" "-"
127.0.0.1 - 1,1,146d55682bc6b69ac082a97d688eb4ca59c14ec084308123e131c2325a89dc59 [29/Nov/2022:17:00:11 +0000] "GET /api/organization?hide_filterinfo=1&limit=100000&order._id=asc HTTP/1.1" 200 179 "-" "-" "-"
127.0.0.1 - 1,1,146d55682bc6b69ac082a97d688eb4ca59c14ec084308123e131c2325a89dc59 [29/Nov/2022:17:00:12 +0000] "GET /api/account/_self/access_hooks HTTP/1.1" 200 11255 "-" "-" "-"
127.0.0.1 - 1,1,146d55682bc6b69ac082a97d688eb4ca59c14ec084308123e131c2325a89dc59 [29/Nov/2022:17:00:12 +0000] "GET /api/account/_self HTTP/1.1" 200 2020 "-" "-" "-"
127.0.0.1 - 1,1,146d55682bc6b69ac082a97d688eb4ca59c14ec084308123e131c2325a89dc59 [29/Nov/2022:17:00:12 +0000] "GET /api/organization?hide_filterinfo=1&limit=100000&order._id=asc HTTP/1.1" 200 179 "-" "-" "-"
127.0.0.1 - - [29/Nov/2022:17:00:36 +0000] "GET /api/account/_self/access_hooks HTTP/1.1" 200 11255 "-" "-" "-"
```

```
127.0.0.1 - - [29/Nov/2022:17:00:36 +0000] "GET /api/account/_self HTTP/1.1" 200 2020 "-" "-" "-"
127.0.0.1 - - [29/Nov/2022:17:00:36 +0000] "GET /api/organization?hide_filterinfo=1&limit=100000&order._id=asc HTTP/1.1" 200 179 "-" "-" "-"
127.0.0.1 - - [29/Nov/2022:17:00:36 +0000] "GET /api/event?extended_fetch=1&limit=10&offset=0&filter.0.aligned_resource%2Fdevice.in=2%2C1&filter.0.event_policy.in=2532%2C2497%2C2516%2C2569%2C2496%2C2524%2C2528%2C2499%2C2487%2C2519%2C2503%2C2577%2C2624%2C2492%2C2526%2C2554%2C2580%2C2514%2C2510%2C2490%2C2565%2C2483%2C2501%2C2563%2C2567%2C2512%2C2571%2C2559%2C2522&order.date_active=desc&order._id=asc HTTP/1.1" 200 2951 "-" "-" "-"
127.0.0.1 - - [29/Nov/2022:17:05:36 +0000] "GET /api/account/_self/access_hooks HTTP/1.1" 200 11255 "-" "-" "-"
127.0.0.1 - - [29/Nov/2022:17:05:36 +0000] "GET /api/account/_self HTTP/1.1" 200 2020 "-" "-" "-"
127.0.0.1 - - [29/Nov/2022:17:05:36 +0000] "GET /api/organization?hide_filterinfo=1&limit=100000&order._id=asc HTTP/1.1" 200 179 "-" "-" "-"
127.0.0.1 - - [29/Nov/2022:17:05:36 +0000] "GET /api/event?extended_fetch=1&limit=10&offset=0&filter.0.aligned_resource%2Fdevice.in=2%2C1&filter.0.event_policy.in=2532%2C2497%2C2516%2C2569%2C2496%2C2524%2C2528%2C2499%2C2487%2C2519%2C2503%2C2577%2C2624%2C2492%2C2526%2C2554%2C2580%2C2514%2C2510%2C2490%2C2565%2C2483%2C2501%2C2563%2C2567%2C2512%2C2571%2C2559%2C2522&order.date_active=desc&order._id=asc HTTP/1.1" 200 2951 "-" "-" "-"
127.0.0.1 - 1,1,146d55682bc6b69ac082a97d688eb4ca59c14ec084308123e131c2325a89dc59 [29/Nov/2022:17:06:10 +0000] "GET /api/account/_self/access_hooks HTTP/1.1" 200 11255 "-" "-" "-"
127.0.0.1 - 1,1,146d55682bc6b69ac082a97d688eb4ca59c14ec084308123e131c2325a89dc59 [29/Nov/2022:17:06:10 +0000] "GET /api/account/_self HTTP/1.1" 200 2020 "-" "-" "-"
127.0.0.1 - 1,1,146d55682bc6b69ac082a97d688eb4ca59c14ec084308123e131c2325a89dc59 [29/Nov/2022:17:06:10 +0000] "GET /api/organization?hide_filterinfo=1&limit=100000&order._id=asc HTTP/1.1" 200 179 "-" "-" "-"
127.0.0.1 - - [29/Nov/2022:17:10:36 +0000] "GET /api/account/_self/access_hooks HTTP/1.1" 200 11255 "-" "-" "-"
127.0.0.1 - - [29/Nov/2022:17:10:36 +0000] "GET /api/account/_self HTTP/1.1" 200 2020 "-" "-" "-"
127.0.0.1 - - [29/Nov/2022:17:10:36 +0000] "GET /api/organization?hide_filterinfo=1&limit=100000&order._id=asc HTTP/1.1" 200 179 "-" "-" "-"
```

The file /var/log/em7/em7ngx.accesslog is configured here:

```
# in the file: /etc/nginx/conf.d/em7ngx_web_ui.conf
    access_log  /var/log/em7/em7ngx.access.log  main;
    error_log   /var/log/em7/ngx.log            info;
```

If you suspected that a particular api URL was linked to a particular sql query that was causing performance problems, you could gather this log, and a slow query log, and analyze the timing to see if they lined up.

post 11.2, with associated queries, EM-43012

You can enable this via system → tools → php developer logs → request URI.

I was talking to Lena one day about this, and she wrote

```
I think the best way to handle this is the new “REQUEST URI” in the PHP logging functionality. https://sciencelogic.atlassian.net/browse/EM-43012
and it adds a line to the mysqli.log that looks like so:
```

```

admin_notifier_get_event_stats()/opt/em7/gui/ap/functions/admin_notifier.inc(line:43) [calling:autofetch_row]
SELECT count(*) AS 'count', MAX(UNIX_TIMESTAMP(date_active)) AS newest_active FROM master_events.events_active ea
 LEFT JOIN sysinfo.uuid_to_mid_to_did m2d ON (ea.Xtype = 1 AND ea.Xid = m2d.did)
 LEFT JOIN master.system_settings_licenses sset ON (sset.id = m2d.mid)
 LEFT JOIN master.policies_events mpe ON (ea.etype = mpe.id)
 LEFT JOIN master.definitions_admin_notifier_events ane ON (ane.event_guid=mpe.event_guid)
WHERE (sset.id IS NOT NULL)
 AND (ane.event_guid IS NOT NULL) LIMIT 1
SIZE: 487 bytes
ROWS:1|ERRNO:0|MSG:
DURATION: 0.00052 seconds
REQUEST_URI: /em7/index.em7?exec=json_admin_notifier_stats

Here’s what an entry from the REST API looks like:
_fetch()/opt/em7/phpshare/classes/system/deepcache.inc(line:76) [calling:autofetch_value]
SELECT val_md5 FROM master.phpcache WHERE cachekey='user/1/keys' AND (ttl=0 OR DATE_SUB(NOW(), INTERVAL ttl SECOND) < last_write) LIMIT 1
SIZE: 137 bytes
ROWS:1|ERRNO:0|MSG:
DURATION: 0.00029 seconds
REQUEST_URI: /api/device/1

1:20
with that, you can cross-reference the nginx.log with the query to determine what IP it’s coming from .
```

This is a much better solution to the problem of investigating API generated queries.

Badly Performing API queries

Sometimes, a customer will have tools that hit the API where one of the queries used by the API backend performs horribly and brings the entire system to its knees. Here is a famous example: Scale Lab CPU Pegged. In situations like that, it is important to be able to block specific portions of the API until the query performance can be improved.

One way to do that is to add a location directive to the CDB file /etc/nginx/conf.d/em7ngx_web_ui.conf, which normally looks something like this:

```
server {
    include  /opt/em7/share/config/nginx.d/em7ngx_web_ui.fragment;

    listen   80 default_server;
    listen  443 ssl http2 default_server;

    if ($nonsecure) {
        return 403;
    }

    access_log  /var/log/em7/em7ngx.access.log  main;
    error_log   /var/log/em7/ngx.log            info;

    ssl_certificate      /etc/nginx/cmscerts/alln1-em7he-db-p03.pem;
```

```
    ssl_certificate_key  /etc/nginx/cmscerts/alln1-em7he-db-p03.key;
ssl_protocols TLSv1.2;
ssl_ciphers 'ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCMSHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-SHA384:ECDHE-RSA-AES256-SHA384:ECDHE-ECDSA-AES128-SHA256:ECDHE-RSA-AES128-SHA256';
ssl_prefer_server_ciphers  on;

    # support for CAC/Client Certificate authentication
    ssl_client_certificate  /var/lib/em7/certs/em7_combined.crt;
    ssl_verify_client       optional_no_ca;
    ssl_verify_depth        2;
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
}
```

By adding a location directive like this, just inside the server block:

```
location ~ ^\/api\/device\/[0-9]+\/aligned_app\/\w+ {
  return 403;
}
```

it is possible to block access to specific API URLs.

Theory

php code: G3code/gui/is/classes/features/ftr_interface.inc

This php file is sometimes used in assembling the queries for responding to API requests. When an API generated query is performing badly, or causing DB contention, it may not be easy to search Bitbucket for large portions of the query. For example, consider this query:

```
19360504        apuser  172.30.8.15:34792       master  Query   3571    Sending data
SELECT COUNT( DISTINCT main.if_id) FROM master_dev.device_interfaces main
        LEFT JOIN (SELECT di.if_id, di.did, IF(di.roa_id=0,ld.roa_id,di.roa_id) as roa_id
        FROM master_dev.device_interfaces di
        INNER JOIN master_dev.legend_device ld ON (di.did=ld.id)
        ) iforg ON (main.if_id=iforg.if_id)
WHERE (iforg.roa_id IN ('449')) LIMIT 1       0.000
```

This was quoted from the output of a process list (see EM-55042: CPU on CDB at 100% constantly, barely able to browse UI CLOSED). I was able to find this by searching through Bitbucket for “main.if_id=iforg.if_id" which revealed this code segment:

```
  protected function _api_idx_args($fields) {
    $args = parent::{__FUNCTION__}($fields);
    $args['table_spec'] += [
      'ld'=>['master_dev.legend_device', 'main.did=ld.id'],
      'iforg'=>[
        '(SELECT di.if_id, di.did, IF(di.roa_id=0,ld.roa_id,di.roa_id) as roa_id
        FROM master_dev.device_interfaces di
        INNER JOIN master_dev.legend_device ld ON (di.did=ld.id)
        )',
        'main.if_id=iforg.if_id'
      ],
```

```
      'org'=>['master_biz.organizations', 'iforg.roa_id=org.roa_id'],

      'tag_map' => ["(SELECT it_map.if_id, tags.tag_id, tags.tag_name  FROM
      master_dev.device_interface_tags_map as it_map JOIN
      master_dev.device_interface_tags as tags ON it_map.tag_id = tags.tag_id
      )", 'main.if_id=tag_map.if_id']
    ];

    $args['pub_to_tablefield']['tag/id'] = 'tag_map.tag_id';
    $args['pub_to_tablefield']['tag/name'] = 'tag_map.tag_name';
    $args['pub_to_tablefield']['device/name'] = 'ld.device';
    $args['pub_to_tablefield']['organization'] = 'iforg.roa_id';
    $args['pub_to_tablefield']['organization_device'] = 'ld.roa_id'; // expose the device's org as a filter field
    $args['pub_to_tablefield']['organization_interface'] = 'main.roa_id'; // expose the interface's org as a filter field
    $args['pub_to_tablefield']['organization/company'] = 'org.company';
    // without one field from the iforg table in the query, the framework will skip this filter.
    // this overrides that behavior, causing a join regardless if a field from the table is referenced.
    $args['base_filters'] += [
      'org_stuff'=>[ 'rawsql', ['main', 'iforg'], NULL, NULL],
    ];

    return $args;
  }
```

Post Mortems

2024-11-04 EM-71013: SL Data Performance API issue

ts_guide_to_jira_link

Link: 💔EM-71013: SL Data Performance API issue CLOSED

From the description:

```
Observed Behavior:
Since the upgrade to Aurora 3, all API queries to retrieve performance data are returning without an associated DID.
Snag_1f8df3a0.png
This makes the API basically unusable for extracting performance data to external sources.

Expected Behavior:
Device IDs should be returned.
Summary of troubleshooting steps taken so far and the current status:
    Placed php into debug and captured logs while making the query:

/api/data_performance/device/dynamic_app?hide_filterinfo=0&duration=1h&beginstamp=1730419200&endstamp=now&presentation_objects=A91C45C0E27A5B7ADDD2E7F50AB9DBFD&rollup_freq=hourly&data_fields=min%2Cmax%2Cavg%2Cstd&filter.0.device.in=49806
Logs attached:
Kyndryl-ELV2-mysqli.log
04 Nov 2024, 02:29 PM
```

```
    Found that the SQL query that returns the data no longer selects the rollup.did field for some reason, which explains why there's no DID in the results:

_get_response_body()/opt/em7/gui/is/classes/features/ftr_data_performance.inc [line:445 calling:query_helper]
SELECT rollup.collection_time, UNIX_TIMESTAMP(rollup.collection_time) 'collection_time',ld.id 'device',rollup.ind 'idx',label.data 'idx_label',min_data 'min_data',max_data 'max_data',avg_data 'avg_data',std_data 'std_data',rollup.pres_id 'pres_id'
FROM dynamic_app_data_1294.normalized_hourly rollup
LEFT JOIN master_dev.legend_device ld  ON (ld.id = rollup.did)
LEFT JOIN master._tmp_pres_to_labelob labelob  ON (labelob.pres_obj_id=rollup.pres_id)
LEFT JOIN dynamic_app_data_1294.app_label label  ON (label.did=ld.id AND label.ind=rollup.ind AND label.object=labelob.label_obj_id)
WHERE ((pres_id IN (4937)))
AND (ld.id IN ('49806'))  AND rollup.collection_time BETWEEN FROM_UNIXTIME('1730419200') AND FROM_UNIXTIME('1730746459')

    I found that CDW US has two stacks on 12.1.2 build 484.
        Stack 1 has been upgraded to Aurora 3 and Stack 2 is Aurora 2.

    The diff between the queries produced by these two stacks clearly shows the issue:
        Snag_1f901807.png

    The Aurora 2 stack returns  Device IDs:
        Query:

/api/data_performance/device/dynamic_app?hide_filterinfo=0&duration=1h&beginstamp=1730419200&endstamp=now&presentation_objects=A91C45C0E27A5B7ADDD2E7F50AB9DBFD&rollup_freq=hourly&data_fields=min%2Cmax%2Cavg%2Cstd&filter.0.device.in=203938
        Snag_1f907f3d.png

    And includes the rollup.did field in the SELECT query:
_get_response_body()/opt/em7/gui/is/classes/features/ftr_data_performance.inc [line:445 calling:query_helper]
SELECT rollup.did, UNIX_TIMESTAMP(rollup.collection_time) 'collection_time',ld.id 'device',rollup.ind 'idx',label.data 'idx_label',min_data 'min_data',max_data 'max_data',avg_data 'avg_data',std_data 'std_data',rollup.pres_id 'pres_id'
FROM dynamic_app_data_961.normalized_hourly rollup
LEFT JOIN master_dev.legend_device ld  ON (ld.id = rollup.did)
LEFT JOIN master._tmp_pres_to_labelob labelob  ON (labelob.pres_obj_id=rollup.pres_id)
LEFT JOIN dynamic_app_data_961.app_label label  ON (label.did=ld.id AND label.ind=rollup.ind AND label.object=labelob.label_obj_id)
WHERE ((pres_id IN (3435)))
AND (ld.id IN ('203938'))  AND rollup.collection_time BETWEEN FROM_UNIXTIME('1730419200') AND FROM_UNIXTIME('1730743454')

Stack 2 Logs:
CDW-Stack2-em7php.log
04 Nov 2024, 02:32 PM

    However, like Kyndryl-ELV2, the Aurora 3 Stack 1 doesn't:
        Query:
/api/data_performance/device/dynamic_app?hide_filterinfo=0&duration=1h&beginstamp=1730419200&endstamp=now&presentation_objects=A91C45C0E27A5B7ADDD2E7F50AB9DBFD&rollup_freq=hourly&data_fields=min%2Cmax%2Cavg%2Cstd&filter.0.device.in=1646386
```

```
        Snag_1f9d624f.png
And it also doesn't have the rollup.did field in the select query:
_get_response_body()/opt/em7/gui/is/classes/features/ftr_data_performance.inc [line:445 calling:query_helper]
SELECT rollup.collection_time, UNIX_TIMESTAMP(rollup.collection_time) 'collection_time',ld.id 'device',rollup.ind 'idx',label.data 'idx_label',min_data 'min_data',max_data 'max_data',avg_data 'avg_data',
std_data 'std_data',rollup.pres_id 'pres_id'
FROM dynamic_app_data_821.normalized_hourly rollup
LEFT JOIN master_dev.legend_device ld  ON (ld.id = rollup.did)
LEFT JOIN master._tmp_pres_to_labelob labelob  ON (labelob.pres_obj_id=rollup.pres_id)
LEFT JOIN dynamic_app_data_821.app_label label  ON (label.did=ld.id AND label.ind=rollup.ind AND label.object=labelob.label_obj_id)
WHERE ((pres_id IN (3333)))
AND (ld.id IN ('1646386'))  AND rollup.collection_time BETWEEN FROM_UNIXTIME('1730419200') AND FROM_UNIXTIME('1730742979')

Stack 1 Logs:
```

2023-02-21 EM-56998 segfaults in php-fpm caused by out of memory

Link: EM-56998: Need to be address Significant segfaults in the /var/log/kern file on TCS DDP EU CLOSED

ts_guide_to_jira_link

In this case, it is possible that the segfaults were caused by the php-fpm process running out of memory. This comment has the details:

```
I asked for advice from Lena on this and she wrote:

hey Chris, i just looked at the log on that escalation:
Allowed memory size of 536870912 bytes exhausted (tried to allocate 131072 bytes) in
/opt/em7/gui/is/classes/api_base/api_response.inc on line 329 (edited)
they probably just need to increase their memory limit?
that’s in the php.ini
support might want to try that
```

2023-02-13 EM-56909 API Long Running Query /api/data_performance_raw/device/dynamic_app

Link: EM-56909: API Long Running Query /api/data_performance_raw/device/dynamic_app CLOSED

ts_guide_to_jira_link

The quoted API path was taking forever to run.

This section describes the code behind these queries: API and NGINX | Badly Performing API queries

2022-12-06 EM-54861 rest api, nginx, ssl, http vs https

Link: EM-54861: rest api post method call from powerflow to sl1 fails CLOSED

2019-07-08 SOL_16338 rest api call generated long running query

Link: SOL-16338: [Escalation] HSBC | API Call generates long running query; Hanging Causing MF Rows Behind CLOSED

From the final comment in that case:

```
We held a call with the customer this morning.

These Dynamic Apps were written by the customer and they generate a lot of short-lived indexes. These indexes are causing issues with the API request's database query returning very slowly.

Their main goal with the DA in question is to track the number of unique wireless users over the course of the month. In order to do so, they are tracking all of the connections and then using a report to gather the data at month's end.

We suggested two possibles solutions:

    Convert to a journal app. This maintains all fo the information in a format that can better be handled by the system but it makes reporting more difficult since
    there is no API for this data. This allows them to keep all of the data that is currently being recorded but will do so in a way that the platform can handle.

    Use the cache api to save the list of unique users. Have the performance app just track the count of unique users thus far. At the month's start, this can reset. T
    his is a viable option because the app aligns to the device so each device can have its own unique key.

Given their goals, we suggested that the cache-api approach would be the best option since it solve their problem directly and would have the best performance.
```